

경구약제 이미지 객체 검출

초급 프로젝트 보고서
고대권, 광민선, 신현수, 윤성현, 이수민
3팀

프로젝트 분석

이번 프로젝트는, 이미지 인식 기술을 헬스케어 분야에 접목해보는 프로젝트를 진행하는 것으로, 사용자가 본인의 모바일 애플리케이션으로 자신이 복용 중인 약의 사진을 찍었을 때, 이미지 인식을 통해 해당 약에 대한 정보를 확인할 수 있는 모델을 만들어야 하는 미션이다.



Fig.1 Pill object detection

사진과 같이, 최대 4개의 알약의 이름(클래스)과 위치(바운딩 박스)를 검출하는 모델을 구현하고, 성능을 지속적으로 개선해나가는 것이 프로젝트의 목표이다. 해당 예시 그림과 같이 끝이 둥근 직사각형(RP) / 원(M) / 원(작은 글씨) / 빨강 타원 (BF) 형의 각각의 알약 모양이나 적혀 있는 글씨 등의 특징을 중점으로 Class 분류 및 위치

를 추정한다.

다음 사항에 대해 [Kaggle 프로젝트](#)는 $mAP@[0.75:0.95]$ 지표를 기반으로 모델의 Object Detection 성능을 측정하고 평가한다. 이는 매우 어려운 조건으로, 객체를 인식하는데 나타나는 경계 상자(Bounding Box)가 실제 위치와 고도로 일치해야 점수로 인정되는 정밀 탐지 성능을 측정하는 것이다. 일반적인 탐지 기준인 $mAP@0.5$ 에 비해 높은 임계값을 요구하므로 모델의 위치 추정(Localization) 오차에 매우 민감하게 반응한다.

즉, 이번 초급 프로젝트는 주어진 입력 데이터의 알약들에 대해 정밀한 Object Detection Model을 구현하기 위해, 데이터의 구성을 및 특성 확인하고, 이에 대한 전처리 방법을 모색한 뒤, 객체를 인식하는데 적절한 모델을 선정하여 훈련 및 테스트를 진행하여 결과를 도출하는 것이 주요 흐름이다.

아이디어 접근

주어진 알약 데이터에 대한 Object Detection을 정밀하게 구현하기 위해, 다양한 아이디어를 기반으로 접근하게 되었다.



Fig.2 Pill data image

해당 사진은 프로젝트에서 주어진 Data 중 일부로, 다음과 같이 각각의 알약의 모양이나 색깔, 표기된 글씨가 다 다르게 나타나 있으며, 객체들에 대한 위치나 사진의 밝기가 전부 세밀하게 다르다는 점을 확인할 수 있었다.

이에 헬스케어 사용자가 사진을 찍었을 때를 가정하여 밝기, 흔들림, 위치, 크기 등의 변경 가능성을 고려하여 데이터를

또한, 이번 프로젝트의 특이 사항으로 원본 데이터는 [Ai-Hub 경구약제 이미지 데이터](#)를 가공하여 제공하였고, 필요시 해당 데이터를 추가로 가져와 사용이 가능하다는 점이다. 그러나 우리 팀에서의 이번 프로젝트 진행은 다음과 같은 조건을 적용하였다. 프로젝트를 진행하는 작은 기업이나 단체가 예산 등이 부족하여 외부 데이터를 추가로 공급받지 못하는 한정적인 상황을 가정하여, 제한된 데이터로 성능을 극대화해 최소 비용으로 최대의 효율을 달성하는 것을 주요한 목표로 선정하게 되었다.

Object Detection 모델을 구현하기 위해, 데이터를 세밀하게 확인하고 검토 작업을 진행한 뒤, 데이터 전체에 대한 증강, 희소 Class 데이터에 대한 부분 증강, 희소 데이터에 대한 Copy-Paste 증강 등 다양한 전처리 기법을 적용해 적은 데이터로 높은 다양성을 나타낼 수 있도록 프로젝트를 진행하기로 하였다.

데이터 EDA

이번 프로젝트에서 주어진 데이터를 확인해 본 결과, 다음과 같은 데이터 구조로 구성되어 있다.

```
Basic-Project/  
└─ Data/  
    └─ raw/                                # 데이터 원본 폴더  
        └─ train_images/                  # 학습용 image: 232(.png)  
        └─ test_images/                   # 테스트용 image: 842(.png)  
        └─ train_annotations/             # 학습용 bbox 좌표, label 등: 763(.json)
```

Fig.3 Pill data structure

Object detection model에 사용할 수 있는 원본 데이터의 구성은 다음과 같이 학습용 / 테스트용 image, 학습용 bounding box 좌표 및 label 정보가 기재되어 있는 annotation 데이터로 구성되어 있다. 구조를 확인해 본 결과, 학습용 image의 수가 테스트 image에 비해 수가 매우 적다는 점을 알 수 있었고, 이를 해결하기 위해 3가지 증강을 적용하여 학습에 사용할 수 있는 image를 다양하게 증가하여 학습에 무리 없도록 조치할 것이다.

또한, train 및 annotation 간의 데이터 일치가 정상적으로 이루어지는지, 확인 작업을 시행하였고, 이에 확인 결과 train - annotation 전부 232개로 일치하여 정상적인 mapping이 가능한 상황임을 인지하였다.

그러나, train image - annotation 간의 bounding box 일치 상황을 확인해 본 결과, bounding box의 위치 오류 / 중복 / 정보 누락의 문제가 나타나게 되었고, 이를 해결하기 위해 데이터 정제 작업을 진행하였다.



Fig.4 Pill image data error

Fig.4와 같이 bbox의 좌표가 올바르게 구성되지 않거나, 중복되어 있고, 또한 해당 annotation 정보가 아예 누락 되어 있는 경우들에 대해 수정 및 데이터 정제 작업을 진행하게 되었다. 해당 오류가 나타나는 train image 및 annotation file을 탐지하고, bounding box 좌표를 추출하여 수정하는 작업을 진행하였다.

또한, 누락된 annotation file에 대해서 해당 약의 label 및 정보들을 그대로 가져오고, 일부 id 등을 세밀하게 수정하며, bbox 좌표 값을 새로 만들어 추가하는 작업을 수행하였다.

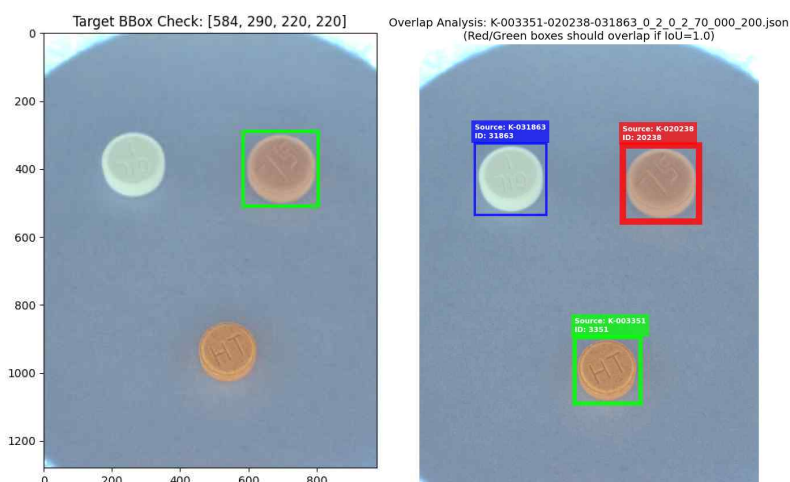


Fig.5 Pill annotation 좌표 생성 및 복구

이후, 수정된 데이터는 다음과 같이 기존 763개에서 771개의 annotation data로 늘어나게 되었고, 이 역시 총 232개의 train - annotation과 일치를 확인하였다.

```
Basic-Project/
└─ Data/
    └─ raw/                                # 데이터 원본 폴더
        └─ train_images/                  # 학습용 image: 232(.png)
        └─ test_images/                   # 테스트용 image: 842(.png)
        └─ train_annotations/              # 학습용 bbox 좌표, label 등: 771(.json)
```

Fig.6 Pill data structure (annotation 수정)

모델 학습을 위해, 수정된 데이터를 기반으로 Train - Validation 분할 작업을 수행하게 되었다. 이때, 각 객체 Class 별 구성 상황은 다음과 같이 나타난다.

클래스(category_id)	객체이름(name)	등장 횟수
3351	일양하이트린정 2mg	157
3483	기넥신에프정(은행업엑스)(수출용)	45
35206	아토젯정 10/40mg	37
16262	크레스토정 20mg)	23
21325	아토르바정 10mg	22
16232	리피토정 20mg 외 1종	21
3832*	뉴로메드정(옥시라세탐)	20
...	...	...
1900*	보령부스파정 5mg 외 3종	15
18357	종근당글리아틴린연질캡슐(콜린알포세레이트)	14
31863	아질렉트정(라사길린메실산염) 외 4종	12
2483	뮤테란캡슐 100mg 외 3종	9
16688	오마코연질캡슐(오메가-3-산에틸에스테르90)	8
19607	스토가정 10mg 외 6종	6
29451	레일라정 외 16종	3

Table.1 Data column information

객체 간 class 개수 및 분포 상황을 확인해 본 결과는 다음 Table.1과 같이 나타나 있다. 총 class는 56종으로, 등장 횟수가 적은 하위 class 객체들이 다수 분포되어 있는 불균형 상황을 확인할 수 있다.

총 train image는 232개로, Train : Valid 비율을 0.33으로 진행해 총 155 : 77로 분할 하였다. 분할 작업에 유의하여, 각 class 종류 당 Data split을 균일한 비율로

분할 하는 MultilabelStratifiedShuffleSplit을 사용하였고, 이를 통해 train 및 valid data에 빠지는 class 객체 없이 전부 균일한 분할 비율을 가지도록 분할 작업을 수행하였다.

분할 하게 된 해당 데이터를 Object Detection에 수행하기 적합한 Model인 YOLO에 적용할 수 있도록, COCO format(.json) 기반의 데이터를 YOLO format(.yaml)으로 라벨 변환을 진행하였다. 이는 좌상단 기준 절대 좌표를 중심점 기준의 정규화를 적용하여 모델 학습에 적합한 학습 데이터로 변형하는 과정이다.

```
Basic-Project/
└ Data/
  ├── raw/                                # 데이터 원본 폴더
  │   ├── train_images/                  # 학습용 image: 232(.png)
  │   ├── test_images/                   # 테스트용 image: 842(.png)
  │   └── train_annotations/             # 학습용 bbox 좌표, label 등: 771(.json)
  ├── yolo_dataset/                      # 원본 split & .yaml 변환
  │   ├── images/                       # image
  │   │   ├── train/
  │   │   └── val/
  │   └── labels/                       # annotation
  │       ├── train/
  │       └── val/
  └── yolo_dataset_aug/                  # yolo_dataset 증강 파일
      ├── images/                       # image
      │   ├── train/
      │   └── val/
      └── labels/                       # annotation
          ├── train/
          └── val/
```

Fig.7 Pill data structure(YOLO format 추가)

데이터 폴더 구조는 다음 Fig.7과 같이 train annotation 수정 후 원본 데이터가 들어있는 raw/ 폴더, data split, yolo label 변환 후 데이터들과 yaml 파일이 들어있는 yolo_dataset/ 폴더, yolo_dataset을 복사하여 팀원들이 수행한 증강 데이터가 들어가는 yolo_dataset_aug/ 폴더로 크게 3가지로 분류된다.

yolo_dataset/ 폴더는 split 이후, train 데이터에 대한 증강 전 원본 데이터로써 사용해 재현성을 갖고, 경로 안정화를 위해 다음과 같이 구성하게 되었고, 이를 복사하여 여러 실험을 반복할 때 삭제와 생성을 자유롭게 하는 yolo_dataset_aug/ 폴더를 구성하게 되었다.

데이터 전처리

데이터 전처리는 다음 3가지를 전부 적용하여, YOLO 모델 학습에 사용하였다.

1. 전체 class에 대한 일반 증강
2. 희귀 class에 대한 일반 증강
3. 희귀 class에 대한 copy-paste

각 전처리 방법에 대해 요약하자면 다음과 같다. 전체 class에 대한 일반 증강은, 색상 변화 계열을 사용하지 않고, 회전 / 스케일 / 밝기 및 대비 / 블러 증강을 적용하고, 전체 class에 대해 한번 증강을 진행하는 방법이다.

희귀 class에 대한 일반 증강은, 전체 class에 대한 증강 조건과 동일하게 수행하며, 다른 점으로는 희귀 class에 대해 조건부 증강을 수행하는 것이다.

마지막으로 희귀 class에 대한 copy-paste는 train image의 빈 공간을 탐색하여 새로운 객체를 복사 및 부착을 시행하여, 새로운 train image를 만들어내는 공간 인지 기반 증강 방법으로, 이 역시도 희귀한 class에 대해 조건부 증강을 수행하여 목표한 수치까지 증가하도록 한다.

1. 전체 class에 대한 일반 증강

먼저, 전체 class에 대한 일반 증강은 다음과 같은 조건들을 적용하였다.



Fig.8 Augmentation settings

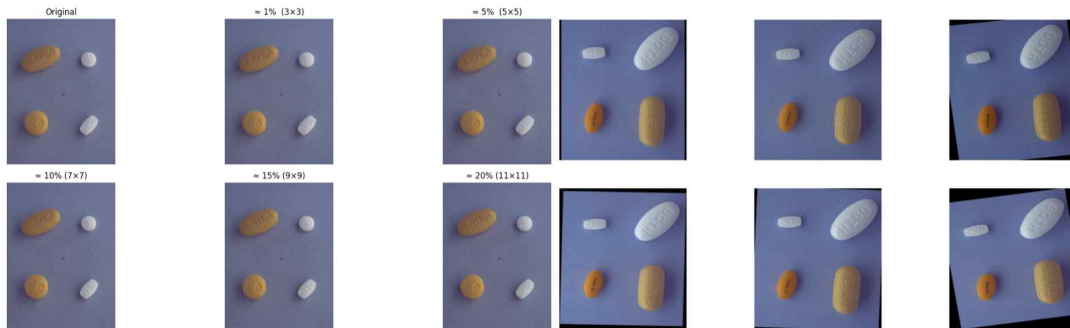


Fig.9 Augmentation example: blur, rotation

알약 데이터는 색상과 각인 정보가 핵심 특징이므로, 정보를 훼손하지 않는 범위 내에서 선택적 증강을 설계하게 되었다. 따라서, 색상 정보를 왜곡하는 증강은 일절 적용하지 않았다.

크기와 회전의 경우 촬영 거리와 각도의 차이에 대한 일반화를 적용하기 위해 일반적인 범주 내에서 사진을 찍을 때 나타나는 범주 안으로 적용하였다. 따라서, Scale은 $\pm 10\%$, Rotation은 $\pm 10^\circ$ 의 범위로 선정하게 되었다.

밝기와 대비는 조명 환경 차이에 대응하기 위해, 적절히 어둡거나 밝은 차이의 범위인 $\pm 10\%$ 범위로 선정하였으며, 가우시안 블러(Gaussian Blur)의 경우 카메라의 초점 흐림이나 미세한 흔들림 상황을 반영하여 blur_limit는 (3, 9): 약 1% 내외의 블러를 적용하였다.

이러한 증강들을 직접 적용해본 결과, 시각적으로 알약에 대한 가독성이 유지되는지 시각화 하였고, 1~2% 수준의 낮은 블러 및 $\pm 10\%$ 범위의 회전 / 크기 / 밝기 등은 각인 가독성이 확인되어 정보 손실이 없는 범위에서 해당 증강들을 랜덤 시행하게 되었다.


```

bp_kwargs = dict(
    format="pascal_voc",
    label_fields=["class_labels"],
    min_area=1,
    min_visibility=0.0,

    sig = inspect.signature(A.BboxParams).parameters
    if "clip" in sig:
        bp_kwargs["clip"] = True
    if "filter_lost_elements" in sig:
        bp_kwargs["filter_elements"] = True
    if "check_each_transform" in sig:
        bp_kwargs["check"] = False

    bbox_params = A.BboxParams(**_kwargs)

```

Fig.10 Albumentations - BboxParams

일반 증강의 핵심은 단순히 이미지만 변환하는 것이 아닌, bounding box의 일관성을 유지하는 것이다. 이를 위해, 이미지 증강 라이브러리인 Albumentations의 BboxParams를 사용해, 이미지를 변형하고 bbox 좌표를 자동 추적할 수 있도록 설계를 진행하였다.

이는 이미지의 변형 후에 바뀌게 된 bounding box 좌표를 Albumentations의 BboxParams가 추적하도록 하며, Albumentations가 $\text{format}(x_1, y_1, x_2, y_2)$ 기준 bbox 변환좌표를 계산하게 되고, bbox가 사라질 때 해당 label도 같이 삭제하게 된다. 만일 변환 후, bbox 면적이 너무 작으면, 제거를 진행한다.

하지만, BboxParam을 사용하더라도 증강 이후 부적절한 bounding box가 남을 수 있는 문제가 발생하게 된다. 예를 들자면, bbox 일부가 이미지 경계를 벗어나거나, 회전·스케일로 인해 bbox 면적이 거의 0에 가까워지는 경우를 의미한다.

이러한 bbox가 학습에 포함되면 에러 없이 학습은 진행되지만, 성능이 저하되는 문제가 발생하게 된다. 따라서 증강을 적용 한 이후, 이미지 경계 클리핑, 최소 면적 기준 bbox 필터링, 비정상 좌표 순서 교정을 수행하는 sanitize 단계를 추가하게 되었다.

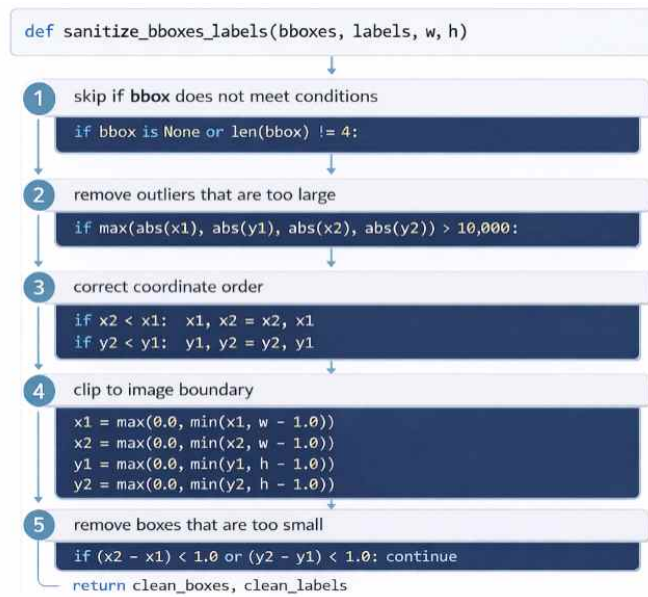


Fig.11 Sanitize algorithm

Sanitize 함수는 다음 Fig.11과 같이 과정 순서대로, bbox 조건을 따로 갖추지 못하면 skip을 진행하고, 너무 큰 이상치가 있으면 제거, 좌표의 순서를 보정, 이미지 경계로 클립을 진행 후, 너무 작은 box를 제거한다.

이 함수는 YOLO 학습이 깨지지 않도록 마지막 안전장치 역할을 하며, 증강 의 실패가 학습 전체의 성능 저하로 이어지지 않도록 한다.

2. 희귀 class에 대한 일반 증강

알약 class 객체들에 대해 직접 분석해본 결과, 다음과 같은 특징이 나타나게 된다.

기준 (Class)	클래스(빈도)	비고
3351(157)과 항상 같이 나옴	16688(8), 31863(12), 18357(13), 18147(15), 32310(15), 13900(15), 22074(16), 41768(16), 20014(17), 33880(17), 19232(17), 38162(18), 3832(20), 21325(22)	ex, 16688) 8번 등장 전부 겹침
3351(157)과 다수 겹침	29667(15/18), 36637(16/19), 20238(14/20), 16232(12/21), 16262(17/23), 35206(19/37)	ex, 29667) 18번 등장 중 15번 겹침
3351(157)과 겹치지 않음	등장이 3번, 6번, 9번, 45번인 모든 Class	항상 겹치지 않음
기준 (Class)	추가사항	비고
3번 등장	12~45번 등장한 클래스와 같은 이미지에 있는 경우 多	클래스 17개 중 12개
6번 등장	12~45번 등장한 클래스와 같은 이미지에 있는 경우 多	클래스 7개 중 5개
8번 등장	12~45번 등장한 클래스와 같은 이미지에 있는 경우 多	클래스 1개 중 1개

Table.2 희귀 Class 분석

Table.2에 따라, 알약 class들의 출현 특징을 기반으로 증강 처리를 진행한다. 이때, 전체 이미지에서 객체가 등장하는 횟수가 9 미만인 모든 class(3, 6, 8번 등장하는 class)를 희귀 class로 구분하고, 전체 이미지에서 객체가 등장하는 횟수가 9~45번인 모든 class를 중위 class, 전체 이미지에서 객체가 등장하는 횟수가 157번인 클래스를 최상위 class로 분류하였다.

여기서 증강 정책의 적용은 다음과 같이 진행된다.

- 일반 증강 조건 및 핵심 기술, 알고리즘은 전체 class에 대한 일반 증강과 동일
- 희귀 / 중위 / 최상위 class가 포함된 이미지 목록을 뽑아, 증강 대상 이미지 목록을 구성한다.
- 중위 class는 희귀 class 증강에 따라, 연쇄적으로 증강될 것이기 때문에, 희귀 class와 한 이미지에 존재하지 않을 때만 조건부로 한번 증강을 진행한다.
- 희귀 class들이 같은 이미지에 나타나게 되면, 더 적게 등장하는 객체를 기준으로 증강 횟수를 결정하도록 한다.

```
FREQ_TO_AUG = {3: 6, 6: 3, 8: 2}, head_catids= {3351},
always_with_3351 = {16688, 31863, 18357, 18147, 32310, 13900, 22074,
41768, 20014, 33880, 19232, 38162, 3832, 21325}
TARGET_RARE_BBOX=10, TARGET_MID_BBOX=18
```

aug_img_set: 희귀 클래스가 포함 된 이미지 목록(증강 대상)
top_img_set: 상위 클래스가 포함 된 이미지 목록(증강 제외)
cond_mid_img_set= mid_img_set- aug_img_set- top_img_set
: 중위 클래스가 포함 된 이미지 목록(조건부 증강 대상)

목표에 도달하지 못한 클래스에 대해서만 bbox를 하나라도 추가해주는지
판단하여 True인 경우에만 증강 저장을 허용한다.

또한, 중위 클래스는 최상위 클래스와 한 이미지에 함께 등장하는 경우가 다수
발견되므로, "최상위 클래스와 함께 등장하거나, 최상위 클래스까지 증가시킬 중위
클래스와 함께 등장하게 되면" 저장하지 않도록 구성함.

만일 3번 / 6번 등장 클래스가 한 이미지에 존재하게 되면,
FREQ_TO_AUG에 따라 증강 횟수를 6으로 결정한다.

이때, 희귀 클래스도 목표에 도달하지 못한 클래스에 대해
bbox를 하나라도 추가 해 주는 경우에만 증강을 허용.

추가로, 8번 등장 클래스는 항상 최상위 클래스와 한 이미지에 나타났으므로,
최상위 클래스 증강은 최대 5장까지만 허용되도록 상한치를 두고 증강을 적용한다.

Fig.12 희귀 class 증강 조건 설정

Fig.12와 같이 세부적인 희소 class 증강 조건을 설정하여, 희귀 class에 대한 일
반 증강을 진행한다. 이를 통해, 알약 객체 간의 학습 불균형을 완화하는 주요한 역
할을 맡게 된다.

이는 모델의 편향을 방지하며, 과적합 문제를 억제하는 효과를 가지게 되고, 결정적
으로 알약 식별 시스템에서 가장 위험하게 판단 될 수 있는 경우인, “중요한 약을 다
른 약으로 착각”하는 상황을 줄여주게 된다. 즉, Recall(재현율)을 높여주어 알약을
놓치지 않고 잡아낼 수 있는 확률을 높이는 역할을 한다.

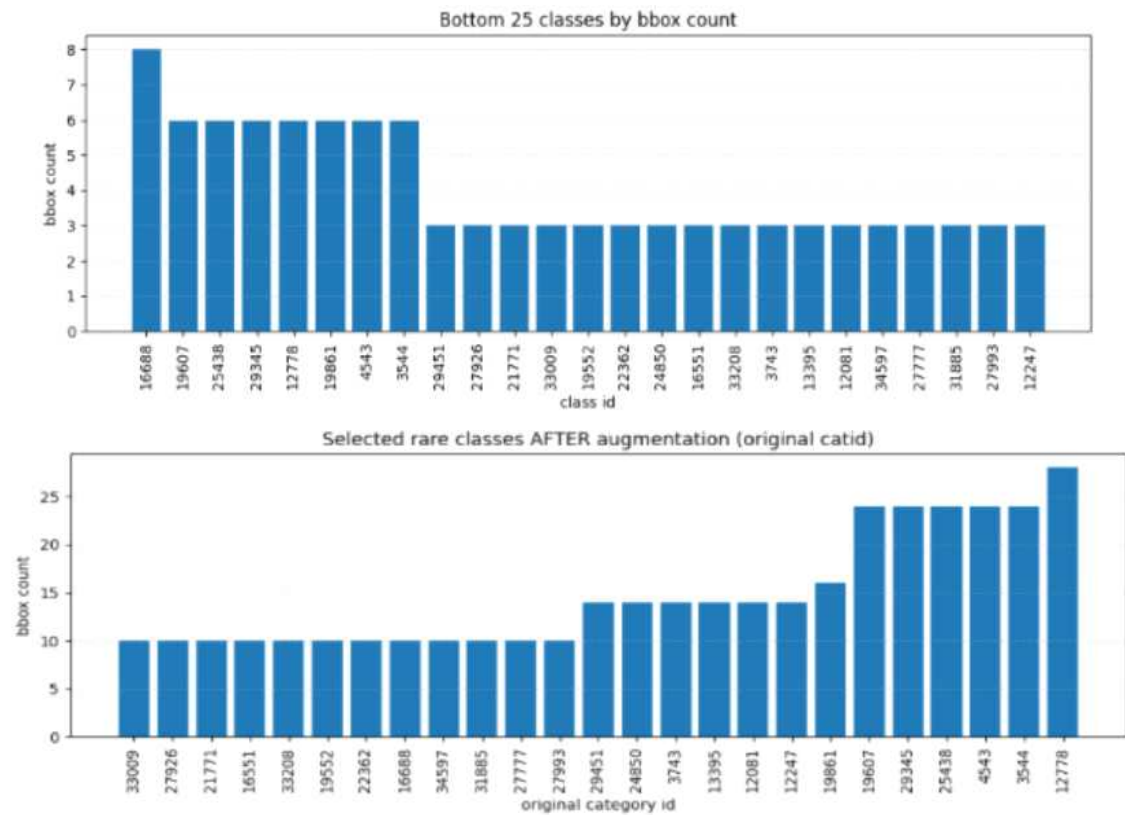


Fig.13 희귀 class augmentation before / after

Fig.13은 희귀 class에 대한 증강이 이뤄진 전 ~ 후 개수를 나타낸 그림으로, 결과적으로 증강 된 이미지는 총 161장이다.

이를 통해, 희귀 class의 등장 횟수가 중위 class 수준으로 증가하여, 데이터의 불균형 비율이 개선되게 되고, 학습의 안정성 증가 및 모델이 희소 class 자체를 noise로 인식하는 것이 아닌, 배워야 할 유의미 한 정보로 인식하도록 변화를 준다.

3. 희귀 class에 대한 copy-paste

희귀 class에 대해서 일반적인 증강을 진행하여도, 그 객체에 대한 정보 다양성이 많이 부족할 수밖에 없다. 따라서, 학습을 진행해도 유의미한 성능 상승을 보기 어려워지는 문제가 발생하게 된다. 이러한 점을 어느 정도 해결하기 위해 도입한 방안이 바로 copy-paste이다.

이전에 시행하던 일반적인 단순 증강들은 기존 알약의 사진들에 대해 배경 안에서만 밝기 / 크기 / 회전 등의 단순한 변형이 일어날 뿐이다. 그러나 copy-paste는 알약 객체를 다양한 배경(다른 약과 함께 있거나, 새로운 위치에서의 알약 탐지 등)에 합성하는 방법으로, 모델이 배경에 의존하지 않고 객체 고유의 특징(알약의 형태나, 각인된 글씨)에만 집중할 수 있도록 한다.

또한, copy-paste 기법을 적용함으로써 희귀 class 객체를 다양한 배경에 노출시키고, 모델이 실제 환경에서 발생할 수 있는 객체 간 중첩 상황에 대한 대응 능력을 비약적으로 상승시킬 수 있다.

이러한 증강 방법을 희귀 class에 적용하기 위해, 타겟 및 금지 class를 다음과 같이 선정하였다.

1 타겟 클래스
반복 실험과정을 통해 과다 증강을 방지하고, 부족한 클래스를 보완하도록 조정한 클래스
- 27777 29451 34597 3743 13395 31885 30308 19861 16688 18357 31863 18147 19232 20014 22074 25367 33880 3832 29667 36637 22347 28763 20877 25469 27733 20238 16232 16262 12247 21771 24850 27926 27993 3544 4543 19607 16548 33009(38개)

2 금지 클래스
배경이미지에서 제외되어야 할 클래스 - 3351, 3483, 16548(3개)

Fig.14 Target / Ban class 선정

Copy-paste 증강을 수행하기 전, 희소 class에 대한 증강 대상과 제외 대상을 Fig.14과 같이 명확히 정의 하였다. 먼저, 타겟 클래스는 반복 실험을 통해 선정한 희귀 class들을 의미하며, 부족한 희귀 class에 대해 필요한 만큼 데이터를 보강하는 것이 목적이다.

금지 클래스는 배경 이미지에서 해당 클래스가 존재하게 되면, 그 배경을 사용하지 않도록 제외하도록 설정하는 것인데, 그 이유는 배경에 해당하는 class 중 출현 빈도가 높은 최상위 ~ 상위 class가 같이 복제되어 원치 않는 분포 폭증이 생길 수 있게 되기 때문이다.

Copy-paste 증강 정책은 다음과 같이 구성하였다.

- deflickt 파악을 통해 타겟 class 별로 최소 목표치까지 부족한 개수를 계산.
- donor 및 base를 선정하여 증강을 적용. 이때 donor는 복사할 객체(알약)을 의미하고, base는 붙여넣을 배경 이미지를 뜻한다. 즉, donor는 같은 이미지 내 다른 box와 거의 겹치지 않는, '깨끗한 객체'를 우선으로 선정하고, base는 금지 class가 포함된 배경을 제외하고 후보를 구성.
- 단순 crop이 아닌, GrabCut 기반 마스크로 객체 가공을 진행해 외곽을 최대한 자연스럽게 다듬고, CLAHE로 대비를 올려 가시성을 확보한다.
- 배치 및 합성을 occupancy 기반으로 빈 공간을 찾고, 가시성의 조건을 주어 배경과 색이 비슷하면(대비가 낮으면) 스킵, 아니면 합성을 진행한다. 마지막으로 feathering을 수행하여 경계를 부드럽게 처리한다.

이러한 과정을 통해, 특정 희귀 class에 대해 부족한 만큼 증강을 진행하며, 깨끗한 객체를 추출해 금지된 배경을 제외, 후보 이미지의 비어있는 공간에 자연스럽게 붙여넣는 작업을 수행한다. 또한 증강 이후, 타겟 별 이미지 생성 수와 전체 class 분포를 확인해 빈도 폭주 문제를 모니터링하여 과한 객체 생성을 막도록 구현하였다.

```

TARGET_MIN_INST = 20, MAX_AUG_PER_CLASS = 100
BAN_BASE_CIDS = {3351, 3483, 16548}, MIN_CONTRAST_DELTA = 8.0

DONOR_MAX_IOU_TH = 0.05, USE_CLAHE_ON_OBJ=True
FEATHER_SIGMA=0.8
    
```

Fig.15 Copy-paste 증강 조건 설정

Copy-paste는 효과가 매우 좋은 증강이나, 그대로 쓰게 되면 부작용이 생기기 쉬워 이를 감소시키기 위한 설계를 다음과 같이 구성하였다.

1	2	3	4	5
base 후보 선정 - base 이미지가 가진 클래스를 set으로 만들어두고 해당하는 클래스가 없는 이미지목록에서만 base 후보 생성 - base 이미지에 있던 클래스도 같이 복제되어 증가하는 문제 방지를 위해 base내 클래스 기준 회소할수록 자주 뽑히게 하는 부작용 감소 설계	donor 후보 선정 - 같은 이미지 내의 다른 박스들과 IoU가 거의 없는 객체만 채택하여 donor 후보 생성 - 각 클래스별로 붙여넣을 "깨끗한 객체 후보 리스트"가 생김	깨끗한 객체 확보 - GrabCut을 활용하여 단순히 사각형으로 자르는 것이 아닌 객체의 외곽선을 따라 마스크 추출 - CLAHE로 객체 명암대비를 높여 배경에 붙었을 때 객체가 흐릿하지 않고 뚜렷하게 보이도록 보정하여 가시성 강화	붙여넣을 위치 탐색 - 후보위치의 ROI가 기존 객체 영역과 겹치는지 검사하여 occupancy 기반 완전히 빈 공간을 탐색 - 가시성(밝기 차이)조건을 주어 배경색과 객체색이 비슷하면 그 위치는 건너뛰고 눈에 보일 만큼의 대비가 있는 곳을 탐색	생성 / 모니터링 - Feathering으로 가장자리만 살짝 흐리게 처리하여 배경에 녹아들게 하고, 객체의 중심은 선명하게 유지하여 객체의 디테일을 유지 - 최종적으로 타겟클래스별 생성된 이미지수, 전체 클래스 분포가 출력되도록 하여 결과 검증 및 모니터링 수행

Fig.16 Copy-paste 부작용 감소 설계

먼저, base 후보 선정에서 배경에 있던 class가 같이 늘어나는 문제를 줄이기 위해, 배경에 포함된 class 분포를 보고, 불필요하게 흔한 class가 더 커지지 않도록 설계를 진행하였다. 이후, donor 후보 선정에선 이미지의 내부에 겹침이 거의 없는 객체만 뽑아 찌그러짐 / 가림 / 마스크의 실패 확률을 낮추었다.

이후, 깨끗한 객체 확보를 위해 GrabCut + 대비 보정을 넣어 붙였는데 안 보이거나, 찌그러지는 문제를 줄여내었고, 위치 탐색에서 배경의 비어있는 공간을 찾는 로직이나 대비 조건을 넣어 겹침 및 가시성 저항 문제를 막아내었다.

마지막으로, 생성 및 모니터링 과정에서 타겟의 생성량뿐만이 아닌, 전체 분포도 같이 확인하여 증강이 오히려 분포를 망치지 않도록 운영하였다.

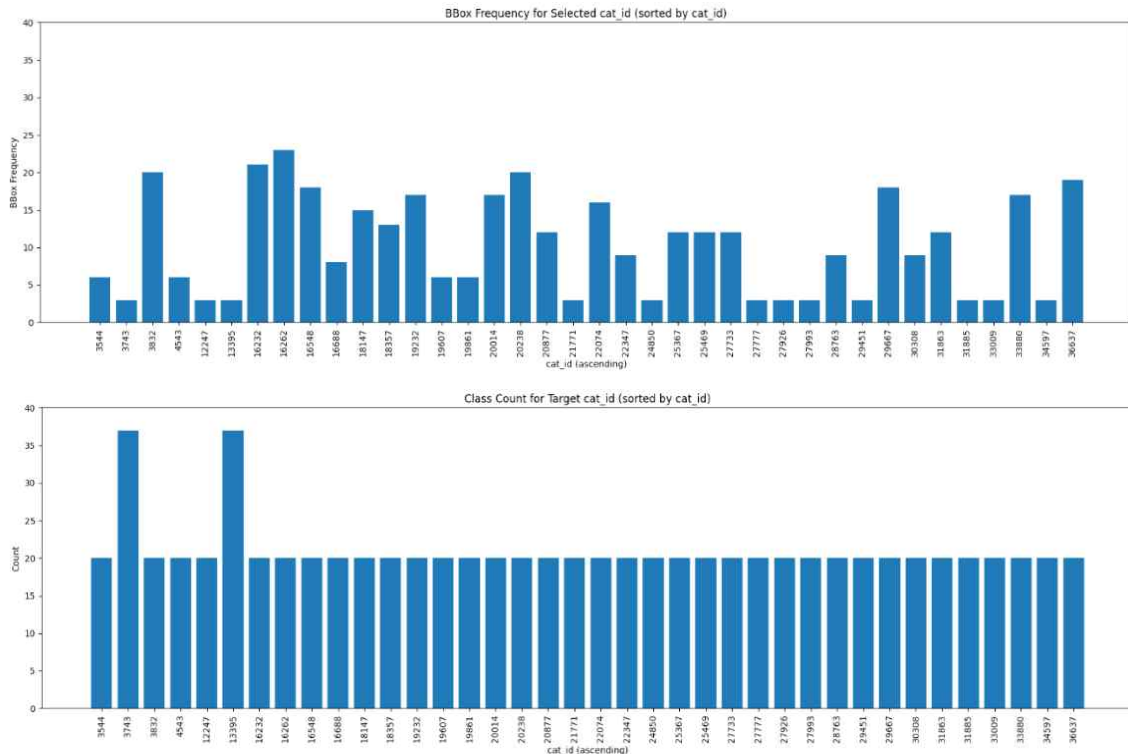


Fig.17 희귀 class copy-paste augmentation before / after

다음은 Copy-paste를 적용하기 전/후 이미지인 타겟 class 객체들의 bbox 빈도 분포 결과로, 총 503장의 이미지가 증강되었다. 증강 전과 다르게 이후에는 매우 적은 타겟 class들이 일정 수준까지 끌어올려지는 방향으로 보강되었고, 이와 동시에 모니터링을 통해 특정 class가 과하게 폭증하지 않도록 확인하였다. 결론적으로, Copy-paste는 타겟 class에 대한 배경 제한, 품질 조건, 분포 모니터링을 고려하여 안정성을 확보한 전처리 전략이다.

모델링

본 프로젝트는 알약 객체에 대한 Object Detection을 수행하기 위해, 이에 맞는 특성과 요구사항에 적합한 모델로 YOLOv11 모델을 선정하게 되었다.

YOLO는 single-stage 객체 검출 모델로써, 한 번의 forward pass로 객체 위치와 클래스를 동시에 예측하므로, 프로젝트에서 제시한 ‘사용자가 본인의 모바일 애플리케이션으로 자신이 복용 중인 알약의 사진을 찍었을 때, 이미지 인식을 통해 해당 약에 대한 정보를 확인할 수 있는 모델’에 적합하다.

YOLO는 Multi-Scale Feature map 기반의 예측으로 작은 객체에 대한 bbox regression 성능이 안정적이며, 한 이미지에 대해 여러 객체가 등장하는 경우인 다중 객체 검출의 상황에 강력한 성능을 보인다. 또한, 자체 라이브러리에 온라인 증강 기술도 탑재하고 있어 증강된 데이터를 학습시키기에 적절한 파라미터를 가진 모델이라고 판단하여 이를 선정하게 되었다.

YOLO 모델 중, 버전 v11을 채택한 이유는 모델의 경량화를 통해 실시간 추론 성능을 확보함과 동시에, 정교해진 특징 추출 능력을 기반으로 작은 알약 객체의 형태 및 각인을 정확하게 식별하기 위해 이를 선택하게 되었다. 특히, 파라미터 최적화를 통해 제한된 자원 환경에서도 높은 정확도를 유지한다는 점이 핵심적인 선정 이유이다.

모델을 사용하기 전, 최종 증강된 데이터를 전부 합친 개수는 다음과 같다.

- 전체 증강 + 희소 객체 증강 + Copy-paste 증강: $232 + 161 + 503 = 896$ 개
- 희소 객체 증강 + Copy-paste 증강: $161 + 503 = 664$ 개

이렇게 다양한 증강 들을 진행 후 한 folder에 합침에도 불구하고 객체 간 class 불균형이 완전하게 해결이 되었다고 보기엔 어렵다. 따라서, Class 불균형 문제를 해소해줄 Mosaic와 Mixup 기법을 사용하게 되었다. 이를 통해 모델이 학습 과정에서 겪는 희귀 class 소외 현상을 알고리즘을 통해 해결하고자 하였다.

Mosaic 기법은, 4장의 이미지를 하나로 합치는 온라인 증강 방법으로, 한 번의 학습 단계에서 여러 class의 객체를 동시에 학습할 수 있도록 한다. 이는 소수 class가 배치(batch) 내에 포함될 확률을 높여 학습의 효율성을 극대화한다.

Mixup 기법은 두 이미지를 선형적으로 겹치게 하여 class 간 경계를 부드럽게 만드는 기법으로, 데이터가 부족한 희귀 class에 대해 모델이 과도하게 Overfitting 되지 않도록 방지하고, 예측의 유연성을 높여 일반화 성능을 개선하였다.

학습을 진행한 이후, 증강 및 전처리만으론 한계가 있는 것으로 보여 Fine-Tuning을 시행하였다. Fine-Tuning은 온라인 증강(Mosaic & Mixup)을 사용하지 않고, 원본 및 오프라인 증강(전체 증강 + 희소 객체 증강 + Copy-paste) 데이터들만을 기반으로 다시 학습을 진행한다.

이후, 모델의 성능을 올리기 위해 최적의 Hyperparameter를 찾아내기 위한 모델 내부 기능인 Optuna를 사용하여 다음과 같은 최적 성능의 Hyperparameter를 찾아내게 되었고, 이를 기반으로도 학습 및 테스트를 진행하게 되었다.

Hyperparameter	Metrics	Hyperparameter	Metrics
lr0	0.01	translate	0.11212
lrf	0.01	scale	0.49992
momentum	0.85437	shear	0.0
weight_decay	0.00026	perspective	0.0
warmup_epochs	2.98601	flipud	0.0
warmup_momentum	0.93041	fliplr	0.48013
box	7.10091	bgr	0.0
cls	0.51753	mosaic	1.0
dfi	1.50138	mixup	0.01237
hsv	0.02615	cutmix	0.0
hsv_s	0.68715	copy_paste	0.0
hsv_v	0.41488	close_mosaic	9
degrees	0.01027		

Table.3 Optuna Hyperparameter

다음은 최적 Hyperparameter를 적용하지 않고, 일반적인 조건들을 전부 적용한 상황에서의 모델 결과이다.

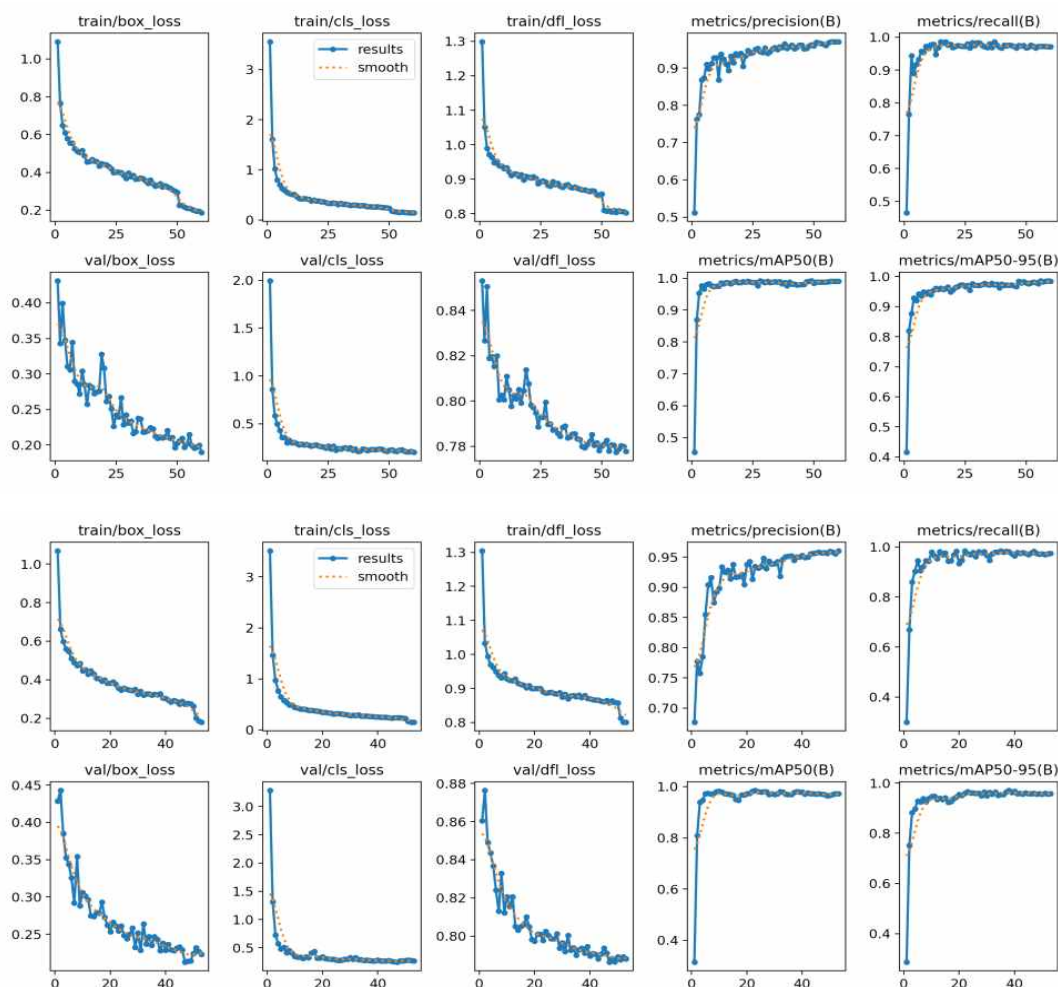


Fig.18 전체 + 희소 + Copy-paste 증강 / 희소 + Copy-paste 증강 비교

적용한 조건들은 다음과 같다.

- Epochs = 60, imgsz(이미지 사이즈) = 640, batch = 16, workers = 2
- dropout = 0.1, mosaic = 1.0, mixup = 0.15
- close_mosaic = 10, patience = 15, augment = True

모델의 학습 및 테스트를 진행한 결과, box_loss, cls_loss, dfl_loss 모두 안정적으로 하향 곡선을 그리고 있다. 특히, Validation Loss가 진동하여 우하향하고 있다는 것은, 과적합되고 있지 않고 원활한 학습이 진행되고 있다는 점을 의미한다.

또한, mAP50 및 mAP50-95 지표가 초반에 급격히 상승 후 고점을 유지하는 점에서, 정교하게 가공된 증강 데이터들이 모델의 초기 학습에 매우 긍정적인 영향을 주었음을 나타내고 있다.

이번엔 최적 Hyperparameter를 적용한 뒤, 위의 일반적인 조건들도 동일하게 전부 적용한 상황에서의 모델 결과이다.

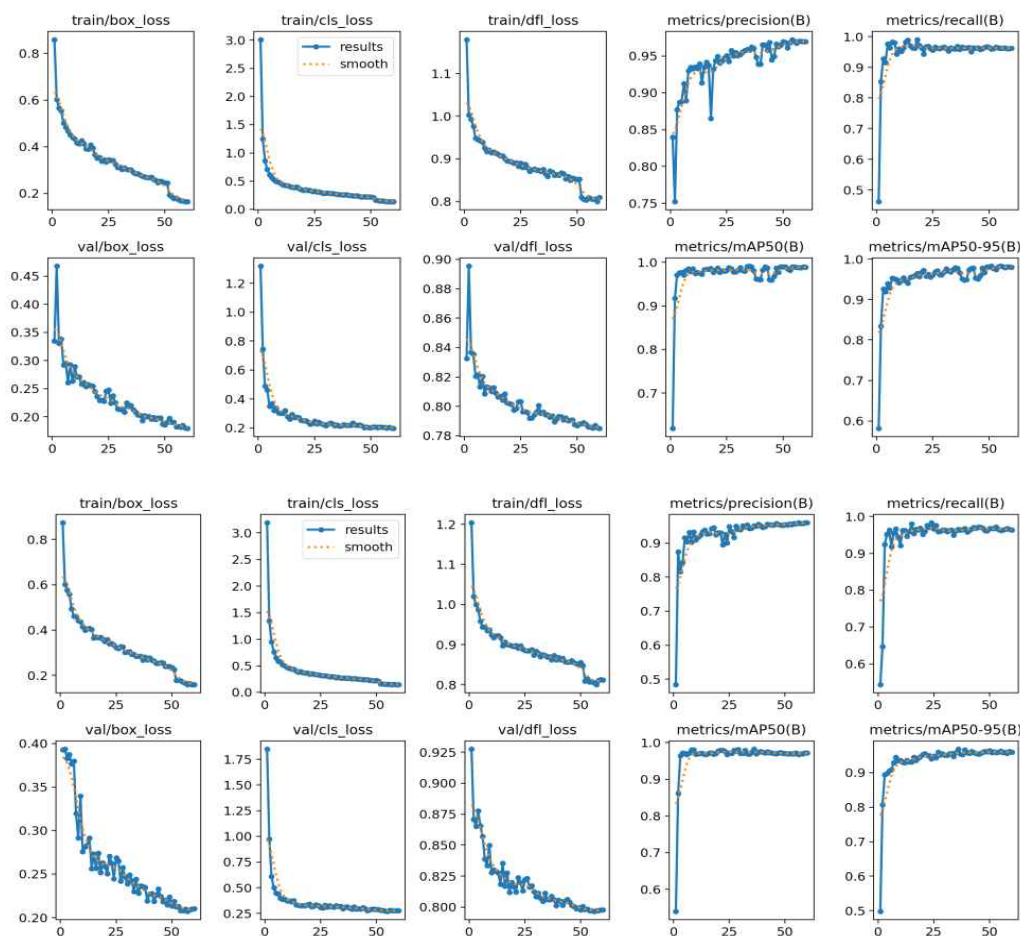


Fig.19 전체 + 희소 + Copy-paste 증강 / 희소 + Copy-paste 증강 비교
(최적 Hyperparameter setting)

최적 Hyperparameter 세팅을 적용 한 결과, 이전과 비교하여 Validation 지표들의 진동 폭이 줄어들고, 매끄러운 곡선을 유지한다. 이는 모델이 학습 데이터에만 과하게 반응하지 않고, 모델 학습 과정이 안정적으로 잘 이루어지고 있음을 뜻한다.

또한, Precision(정밀도)가 95%(0.95) 선에 가깝고 빠르게 안착하며, mAP50-95의 지표가 좀 더 향상되었다. 이를 통해, 높은 정밀도와 재현율을 이끌어 낼 수 있음을 확인하였다. 이때, 가장 성능이 좋은 전체 + 희소 + Copy-paste 증강을 Kaggle 평가 기준에 따라 평가한 결과 Public Score: 0.95581을 기록하게 되었다.



Fig.20 전체 + 희소 + Copy-paste Test 결과 시각화
(최적 Hyperparameter setting)

다음은 Test 결과를 시각화한 것으로, 일부 희소 객체는 중복되어 인식되는 문제가 발견되나, 대부분 알약의 Object Detection 성능이 높게 나타나고 있어 객체 분류 및 위치 추정이 원활하게 이루어지고 있음을 확인할 수 있다.

결론 및 고찰

모델 학습 결과, 적은 데이터를 기반으로 증강 및 전처리 기법을 사용해 모델의 성능을 확보할 수 있음을 확인하였다. 비용이나, 시간과 같은 현실적인 문제에 직면했을 때, 가지고 있는 적은 데이터만으로 이미지 증강 전략을 적절히 시행한다면, 모델의 성능을 더 좋게 개선할 수 있었다. 따라서, 제한된 상황에서도 object detection의 실현이 가능함을 나타내었다.

다만, 단순 증강만으로는 데이터의 다양성이 충분히 확보되지 않아 일반화 성능 향상에 제한적이고, 모델 성능을 극대화하는 데에 한계가 존재한다. 결론적으로, Object Detection을 수행하는 프로젝트를 진행할 때 각 목표와 상황에 맞는 세밀한 전략 구상이 필요함을 인지하였고, 이에 적절한 타협점을 찾는 것은 큰 도움이 된다는 점을 배우게 된 시간이었다.